Component-Based Methodology for Hardware Design of a Dataflow Processing Network

Robert Grou-Szabo, Hany Ghattas, Yvon Savaria and Gabriela Nicolescu
Electrical Engineering Dept., Microelectronics Research Group
École Polytechnique, P.O. box 6079, succ. Centre-Ville Montréal, Qc. Canada, H3C 3A7
{grou, ghattas, savaria}@grm.polymtl.ca, gabriela.nicolescu@polymtl.ca

Abstract

This paper proposes a new methodology for the design of reusable IP blocks used in data-flow architectures. These IP blocks are elaborated by encapsulating individual operators that constitute part of an algorithm within wrappers that possess a configurable communication layer. These IPs communicate using the VCI protocol, and the interconnections are automatically generated from a Data Flow Graph. The interface wrapper has been designed and simulated in 0.18um CMOS technology. When implemented using 2 10-bit input ports, a 12-bit output port and a FIFO depth of 8, synthesis results show that the circuit has a gate count of 1730 NAND gates with a maximum operating frequency of 400 MHz before placement and routing.

1. Introduction

Due to the increasing density and complexity of circuits, the use of component-based design methodologies is quickly gaining popularity [10]. These methodologies allow for components of a system-on-chip (SoC) to be assembled much like ICs are interconnected in a board-level design. Component-based methodology may be applied at two different levels of abstraction during the design of a complex SoC. At the first level, complex IPs are created by assembling existing components, and at the second level, the obtained IPs are composed to design the SoC. Our work is situated at the first level, where complex IPs are designed. We propose a methodology enabling automatic integration of existing components for the efficient design of hardware IPs. The key issue is the definition of generic wrappers able to interconnect a set of given components.

When developing a streaming data application (ex: video, DSP or network processing), designers have limited options when deciding where components of an IP will come from. The two most widely used sources of IP subcomponents involve either using components generated by a behavioural compiler or reusing pre-verified blocks. Although the use of behavioural compilers offers preliminary results very quickly, it generally implies a high overhead content. Moreover, the generated components can never be optimized for every application, and the interfaces are rarely configurable. The second type of source for IP components, consisting of reusing ready-made components, can reduce design time, but usually requires that the various interfaces of recycled components be matched. For this reason, the need to separate a component's behaviour from its interface becomes evident. To facilitate reuse, the design of a component's interface should be flexible, configurable and tailored to suit the chosen class of application. These configurable interfaces generally consist of several layers of command and control logic that increase latency. However, in a typical dataflow application, such as multimedia processing, a high throughput is usually desired, as opposed to a deadline-oriented design. Thus, latency can be inserted without adversely affecting the system performance. In many cases, a latency of several clock cycles will not even be noticed. Therefore, the addition of a high-latency interface may not constitute a drawback in many data streaming applications, and the choice of a component-based design style may be justified.

In this paper, a component-based design methodology for data flow processing networks is proposed. The methodology generates synthesizable VHDL code using an untimed representation of a dataflow graph as the source. The code generation phase involves the use of a flexible interface wrapper with configurable data ports linked to a configurable communication channel. A graphical interface was also developed to complement the VHDL interface

Proceedings of the 9th International Database Engineering & Application Symposium (IDEAS'05)
1098-8068/05 $20.00 © 2005 **IEEE**

wrappers. It is useful for specifying a desired network of interconnected components and it also assists with the configuration phase of each component's generic parameters.

Section II summarizes related previous work. The design methodology followed in this work is presented in Section III and its analysis is presented in Section IV. Section V discusses the VHDL implementation and conclusions are drawn in Section VI.

2. Related Work

Several previous papers propose component-based design methodologies. Some of them deal with assembling existing components for complex IP design and others with combining the obtained IPs, at system level, to design the SoC. All of these contributions face the same challenges, the main difference being at the communications interface level.

The separation between the functionality of an IP and its interfaces has previously been proposed in [5], as well as a generic architecture for hardware/software interfaces. It allows designers to focus on the behaviour of individual blocks without worrying about how they will be connected.

A well-known bus architecture that applies separation of behaviour and communication interfaces is the Open Core Protocol (OCP) bus [2]. However, the use of an OCP bus in a simple point-to-point application is not practical, due to its high complexity.

In [3], a wrapper-based bus architecture is proposed. Unlike many other buses, it does not require embedded memory elements to buffer incoming requests. The main drawback of this architecture is that it does not support multi-rate streams. (i.e. two data streams that converge in a processing node, each possessing a different streaming behaviour, may not be handled correctly). To overcome this problem, FIFOs are well suited to synchronize converging multi-rate data streams. Nevertheless, the area occupied by memory elements generally needs to be minimized. Thus, new techniques to optimize their capacity without hindering throughput are needed [4].

A methodology to automatically generate interfaces between hardware subsystems is presented in [6]. Since the proposed design is multipoint and allows shared resources, an arbitration scheme is needed. By contrast, a point-to-point communication protocol is sufficient to implement communication channels in a computational model that processes unidirectional data streams. It provides several types of operators and functionalities with the interface and can assist with the design of a multiprocessor network on chip, or even support automatic generation of assembly code according to the sequence of operations in a DSP application. However, the tool remains essentially a software platform, and, by contrast, this paper mainly focuses on designing specialized hardware RTL co-processors.

One of the two possible approaches that can be used to implement a hardware circuit is to map it out from a dataflow graph (DFG). The control system can either be centralized [11] [12] or distributed [13]. A distributed approach is used in our methodology, as it is more flexible and scalable.

This work is based on an architecture first proposed in [7]. In that paper, the approach was benchmarked with a single complex DSP application that was implemented as a single pipeline, and in its previous implementation, the architecture did not support algorithms expressed along multiple parallel data processing branches.

Compared with previous work, this paper proposes an architecture with improved flexibility that extends the range of suitable applications. A graphical user interface is also provided to assist the designer in the task of efficiently capturing the application at a high level of abstraction.

3. Design Methodology

The design of a processing network can be derived from several types of models of computation such as Data Flow Graphs (DFG), Petri Nets or Finite State Machines (FSM).

The proposed methodology supports mapping the DFG representation of an algorithm to a hardware implementation of the processing chain. This chain is composed of processing modules, expressed in VHDL, with configurable interface wrappers. The data dependencies and dataflow expressing the processing algorithm are derived through the use of a graphical user interface. The rest of this section presents the methodology and the concepts behind it. We use the same definitions and terminology presented in [9].

A dataflow graph is a model of computation consisting of processing nodes and communication channels. The nodes communicate by transferring tokens through unidirectional data channels.

A processing node is a behaviour encapsulated within an interface wrapper. This behaviour is an elemental component or an operator of the processing algorithm being implement and can be of varying degrees of complexity. The wrapper interface is responsible to control the flow of data to and from an operator.

A communication channel is the link connecting nodes, allowing information exchange. In the model, the communication channels are configurable templates written in VHDL. They implement the protocol through which operators exchange information. These templates are flexible and take into account various kinds of data types. Thus, any input port of a processing node will accept data from an output port of any other node, including its own (feedback loop). Moreover, as further discussed in section 4, this generic connectivity allows dynamic reconfiguration of interconnections using multiplexers to reroute data tokens as needed. Due to physical restrictions of design dimensions, unbounded channel buffering is impossible. Furthermore, without suitable protocols resending data as assumed here, discarding unprocessed data is not an option, since it can render a signal unintelligible. For these reasons a communication channel is defined as being bounded by a predetermined depth. The number of unconsumed data tokens on a channel will not exceed this depth. At runtime, processing nodes consume and produce data tokens on their communication channels.

A data token is a packet of information being processed and consists of three types of information bundled together. First is the data packet, the result of an operation or data value needed for subsequent processing. The second type of information contained in a data token is the validity status of the token. To keep a pipeline full, it may be necessary for a processing node to generate an invalid token, thus the data contained within the token contains no valid data and is considered to be corrupted. Alternately, some real-time systems, such as cameras (CCDs), must produce data packets according to strict deadlines, and when data is not available by that deadline, they will produce invalid data packets to help maintain system synchronisation. Any subsequent data tokens dependant on a corrupted token will have its valid status changed and will also be considered corrupted. The third type of information contained in a data token is the sign encoding method for the data packet, such as two's complement, sign and magnitude or unsigned data.

Depending on the implemented algorithm, different numbers of tokens are produced and consumed at each triggering of a node. This is referred to as a node's firing rules. In our model, two rules must be met before a node will fire: (1) at least one data token must be present on each input channel, and (2) there must be an empty slot on every output channel ready to accept the result after the processing node has fired. When a communication channel reaches its storage capacity, only the node feeding that channel will be halted until a free slot is available. Nodes consuming tokens from a full channel operate normally. For instance, in Figure 1 the processing node C is stalled while waiting for a data token from node B. During this time, nodes A and D are free to operate.

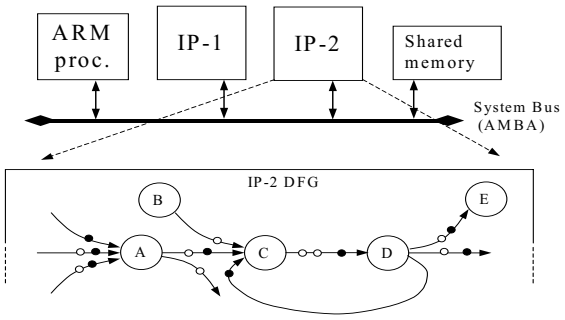

Figure 1 Dataflow process network; token exchange diagram. A black dot represents a valid data token, a white dot is an empty buffer slot

Intuitively, one may think that the slowest operator in a processing chain will limit throughput. However, other factors can influence the propagation of data, such as traffic congestion, bursty data flow and the relative behaviour between producer and consumer of a data token. As a result, throughput can often be limited by the transmission of data rather than by the computation time. Thus, an important advantage of the component-based approach, to divide a complex behaviour into simpler processing nodes, is that even when one node stalls, all the other nodes for which the firing rules are respected are free to continue working. However, the difficulty is to assemble the processing nodes and to guarantee their correct synchronization.

To overcome this synchronization problem, a processing node was defined and implemented. Its internal architecture is illustrated in Figure 2-(a). A node with 2 inputs and 2 outputs is illustrated. As we can see in this figure, the interface is composed of four types of modules:

- *The FIFO control* acts as the consumer on a channel and communicates with preceding nodes if the channel is full.
- *The Output control* is the producer on a channel and it writes data tokens onto a node's subsequent channels.
- *The Operator* implements the functional behaviour of a node, consuming data tokens.
- *The Top Control* coordinates the sub-components together when firing occurs, enabling the operator when enough tokens are present, signalling to the output control that a token is ready after processing

has finished, and signalling the FIFO control to delete a token after it has been consumed.

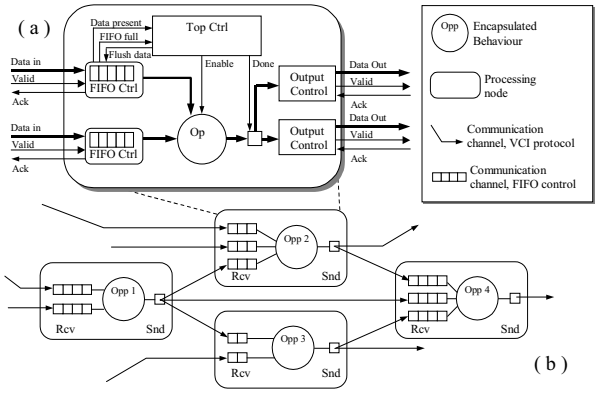

Figure 2 (a) Typical internal architecture of a processing node (b) Processing nodes and communication channels interconnections

4. Improving Efficiency

This section proposes several means to preserve and improve the efficiency of the proposed methodology. The covered issues relate to data representations, granularity, and configurability of the modules and their data paths.

The overhead is one of the main concerns when designing a "wrapper". Sources of overhead include the added costs in time, area, manpower or any other resource (such as power consumption or bandwidth). Deciding how to partition an algorithm has obvious impacts on the overhead and performance. A cost/benefit analysis using granularity as a metric will help reduce costs.

To determine the granularity of the operators in a design, two points need to be considered: the desired abstraction level and the target class of applications.

- The coarser the chosen operator granularity, the less benefit we obtain from having a distributed architecture. Conversely, the finer the granularity is, the greater the overhead. A balance between operator size and performance must be struck

- Traditional methods of grouping a class of applications based on the target market are no longer viable. Instead, a class of application is better defined by the behaviour of the data stream and the way data is processed.

Because communication channels are event-driven, they can operate in separate clock domains from the processing nodes. Consequently, each node operates independently. The result is a Globally Asynchronous Locally Synchronous (GALS) system and each node can have its own clock. An operator performing compute intensive computations could be driven at a faster clock speed, while less complex operations could be bundled together within the same node to keep it busy while possibly operating at a slower clock speed. Power efficiency can be increased by dynamically putting a process node in sleep mode while waiting for data tokens. Combining slower operators and reusing resources can also minimize area. Because the communication channels are configurable, they can connect any output port with an input port from any node. An interesting benefit to this architecture is that multiplexers can be inserted between nodes to redirect the communication channels and modify the data path at runtime. Modifying the control signals of the multiplexers can configure which algorithm is executed. This is illustrated in Figure 3, where the modules' respective outputs can have multiple destinations. In all cases, connections between modules remain point-to-point connections. Thus extra functionality can be added to an existing design without affecting proven ones.

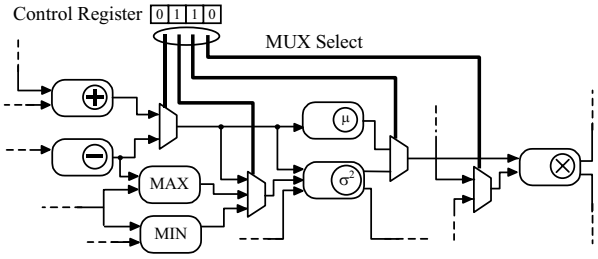

Figure 3 Multiplexer insertion between processing nodes to dynamically redirect communication channels and reconfigure the design

To keep the design of the processing nodes configurable and reusable, all communication channels are considered to transmit data in fixed-point precision. However, the position of the decimal point is allowed to change between nodes to make producers and consumers compatible, without unduly increasing complexity of the processing modules. This does not prevent processing nodes from performing floating-point operations if needed. The VHDL wrapper is not responsible for aligning a data token with an operator before it is processed or transmitted; this alignment must be hard-coded into the operator when it is being encapsulated. A similar problem arises for signed tokens that are consumed by a node expecting an unsigned token.

Several parameters were left generic in our design, such as the number of inputs, outputs, bus widths,

FIFO depth and their "almost full" values, the type of synchronization (synchronous or asynchronous data transfers) on a channel and finally rising edge or falling edge sensitivity. In our approach, we assume that data flows are unidirectional. Furthermore, to simplify the design and reduce overhead, a simple point-to-point protocol was needed to implement the communication channels. For these reasons, the VCI protocol was chosen to act as an interface between different processing nodes. This kind of "wrapper interface" behaves much like a bus with a single master interface connected to any number of slaves. Figure 2-(b) illustrates the network's connectivity and communication channels. The clock speed of a point-to-point link can be designed to execute much faster than the internal clock speed of a processing node. Consequently, a given node's wait time will normally be due to an upstream node's processing time rather than the time it takes a token to be transferred.

The interface of a processing node transferring a data token must know when the data is ready to be transferred. Two situations can occur to indicate when the processing of a data token has finished. Either the latency is known before processing or it is data dependant. When the latency is fixed or cyclic, the number of clock cycles to process a token must be provided for the interface whether the operator is pipelined or iterative. In the case of a data-dependant operator, the operator itself must supply the data valid signal each time it has finished processing a token.

5. Implementation and results

A first implementation of a Wiener video filter was programmed in VHDL to serve as a reference benchmark. The design complexity is given on the first row of Table 1. Then the same filter was programmed using the proposed component-based approach.

The second row of Table 1 represents the complexity with the algorithm segmented into 7 components. On the third row, the complexity of a blank processing node's interface is given. After synthesis using TSMC's 180nm CMOS technology, a design complexity of approximately 1730 gates is obtained with a maximum operating frequency of 400MHz. Synopsys's Design Compiler was used for Synthesis. The "worst case" libraries were used and optimized for processing speed rather than area. The second column of Table 1 represents the same designs compiled for use with an Altera FPGA.

Table 1 ASIC Vs FPGA Implementation of Wiener video filter and Wrapper Interface

Configuration	Compiled using TSMC, worst case library	Compiled for Cyclone II FPGA
Wiener filter, no wrappers	~18,200 NAND gates	6873 Logic elements
Wiener filter, 7 encapsulated subcomponents	~30,000 NAND gates	9525 Logic elements
"blank" wrapper, 2 x 10-bit inputs, 1 x 12-bit output, FIFO depth of 8	~1,730 NAND gates	874 Logic elements

A Graphical User Interface to capture component based network structures was also implemented in C# (Figure 4) using the .Net framework and a graphic library called Netron [14]. The graphic interface allows a user to plot the data dependency network and configure the generic parameters of each node. Once a graph is complete, the GUI generates the top level VHDL file. This GUI is composed of three work areas:
- the library of existing components available to assemble and construct an algorithm (see section A in Figure 4).
- the windowpane (see section B in Figure 4): this is the work area where the algorithm is assembled using components dragged and dropped from the library.
- an interface to capture the generic parameters of the component currently selected in the windowpane (see section C in Figure 4). Manipulating these parameters modifies the generated VHDL file.

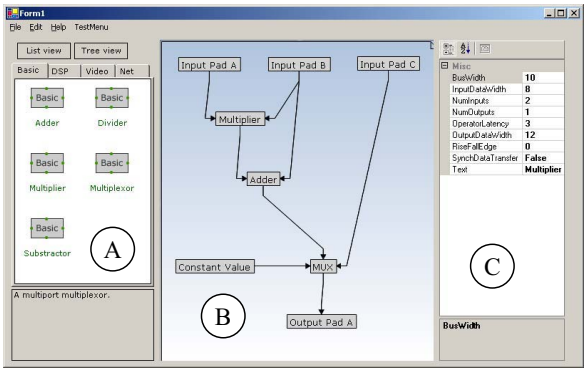

Figure 4 Graphical interface, NeFilter, used to draw a DFG and configure a node's generic parameters

Without a user interface, the algorithm needs to be coded by hand. In a complex algorithm, the number of interconnected signals increases rapidly with each additional processing node. Consequently, a graphic

interface greatly speeds up the process and some functions can be automated. Moreover, inaccuracies and mistakes can easily be introduced in a hand written program. By contrast, an automated process, if done properly, will not create anomalies in a program. In addition, generic properties of a given node have to be matched with those of every other node linked through a shared communication channel. The GUI also integrates a verification tool that can automatically validate these matching parameters and warns the user when a mismatch occurs. The speedup factor in design time can be observed when the Weiner filter was first programmed in VHDL and took several weeks to complete, whereas using the proposed methodology of and graphical interface the same task takes a matter of hours.

6. Conclusion and Future work

A component-based methodology was developed to generate synthesizable VHDL from a dataflow graph. The result is a reduction in design time and the creation of custom IP cores is facilitated. By separating a component's behaviour from its interface and by using configurable templates to generate the control and interfaces of a processing network, engineers can concentrate on designing the behavioural aspect of a system. An important characteristic of this methodology is its flexibility: functionality can be added or removed with minimal modifications. To control data transfers between processing nodes, the VCI protocol was used. A moderate complexity benchmark composed of 7 processing nodes was implemented in VHDL and synthesized. An empty node 2 10-bit input ports, a 12-bit output port, and with a FIFO depth of 8 was also synthesized. The resulting design complexity is approximately 1730 NAND gates when synthesized with TSMC's 180nm CMOS technology. A GUI was programmed in C# to facilitate configuring the processing nodes and generate VHDL files.

7. References

[1] (2001) Virtual component interface specification Version 2 (OCB 2 2.0). Virtual Socket Interface Assoc. [Online]. http://www.vsi.org

[2] (2003) Open Core Protocol Specification Rev 1.1.1 OCP International Partnership (OCP-IP). [Online]. http://www.ocpip.org

[3] K. Anjo, A. Okamura, M. Motomura, "Wrapper-based bus implementation techniques for performance improvement and cost reduction", IEEE Journal of Solid-State Circuits, Vol. 39, pp. 804-817, May 2004.

[4] V. Chandra, A. Xu, H. Schmit, L. Pileggi, "An interconnect channel design methodology for high performance integrated circuits", Proc. D.A.T.E., Vol. 2, pp. 1138-1143, Feb. 2004.

[5] S. Yoo, G. Nicolescu, D. Lyonnard, A. Baghdadi and A. A. Jerraya, "A generic wrapper architecture for multi-processor SoC cosimulation and design," Proc. 9th Int. Symp. Hardware/Software Co-Design, pp. 195–200, April 2001.

[6] J. Smith, G. DeMicheli, "Automated composition of hardware components," in Proc. D.A.C., pp. 14–19, June 1998.

[7] L.-P. Lafrance, Y. Savaria, "A framework for implementing reusable digital signal processing modules", Proc. 4th IEEE International Workshop on System-on-Chip, pp. 51-54, July 2004.

[8] (2005) The Ptolemy project, Dept. of Eng. and computer science. Berkley University [Online] http://ptolemy.eecs.berkeley.edu/

[9] E. A. Lee and T. M. Parks, "Dataflow process networks", Proceedings of the IEEE, May 1995.

[10] ITRS, "International technology roadmap for semiconductors, Design", Edition 2004.

[11] J. Horstmannshoff and H. Meyr, "Efficient building block based RTL code generation from synchronous data flow graphs", Proc. 37th Design Aautomation Conference, pp. 552-555, June. 2000.

[12] H. Jung, K. Lee, S. Ha, "Efficient hardware controller synthesis for synchronous dataflow graph in system level design", IEEE Trans. On VLSI Systems, vol 10, pp. 423-428, Aug. 2002.

[13] M. Ade, R. Lauereins, J.-A. Peperstraete, "Hardware-software codesign with GRAPE", 6th IEEE International workshop on Rapid System Prototyping, pp. 40-47, June 1995

[14] (2005) The Netron Project, François Vanderseypen and Lutz Roeder, Sourceforge repository [Online] http://netron.sourceforge.net/ewiki